

Username	Role	Status	Last change
adamru	Reviewer	● LGTM	Apr 15, 2026
fabienbp	Reviewer	● Pending	Apr 15, 2026
lindanyberg	Reviewer	● Pending	Apr 15, 2026
litianma	Reviewer	● Pending	Apr 15, 2026
whua	Reviewer	● Pending	Apr 15, 2026

 Approval Instructions: Please approve or LGTM through the [G3 Assist](#) sidebar.
For more information, see [go/g3a-approvals-reviewing](#)



Developer’s Guide to Gemini Live Avatar (Preview)

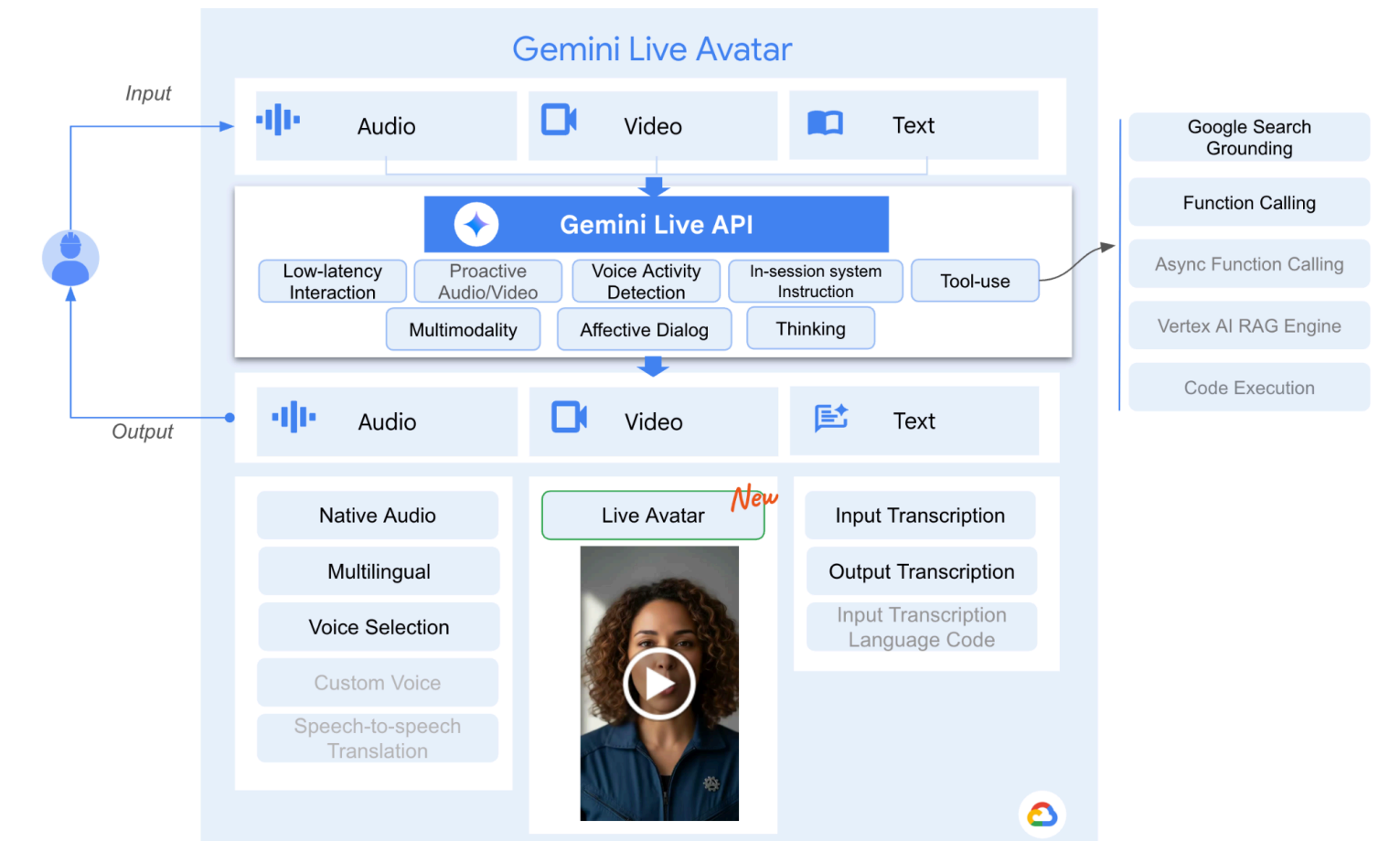
Eric Dong Apr 16, 2026

Legal Terms

This product is a Pre-GA offering, subject to the "Pre-GA Offerings Terms" in the General Service Terms section of the [Service Specific Terms](#). For this generative AI Preview offering, Customers may elect to use it for production purposes, and may process personal data as outlined in the [Cloud Data Processing Addendum](#), subject to the obligations and restrictions described in the agreement under which you access Google Cloud. **Pre-GA products and features may have limited support, and changes to pre-GA products and features may not be compatible with other pre-GA versions.** For more information, see the [launch stage descriptions](#).

Overview

The **Gemini Live API** enables low-latency, bidirectional voice and video interactions with Gemini. The **Live Avatar** feature takes this a step further by synthesizing a video of an avatar speaking the response, rather than providing just an audio response. You can choose from a set of built-in avatars and voices, or provide your own custom avatar image and voice sample.



[Quick Start via Vertex AI Studio](#)

[API Configuration & Implementation](#)

- [Enabling Live Avatar](#)
- [Configuring the Avatar](#)
- [Configuring the Voice](#)

[Implementation Examples](#)

- [Using WebSocket](#)
- [Using Gen AI SDK](#)

[Important API Changes](#)

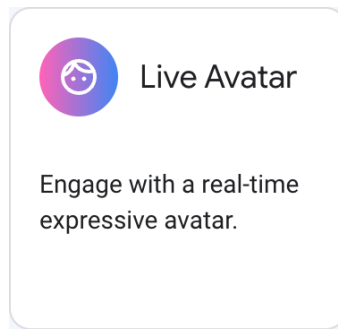
Quick Start via Vertex AI Studio

For rapid testing and evaluation, follow these steps to configure a Live Avatar session through Vertex AI Studio:

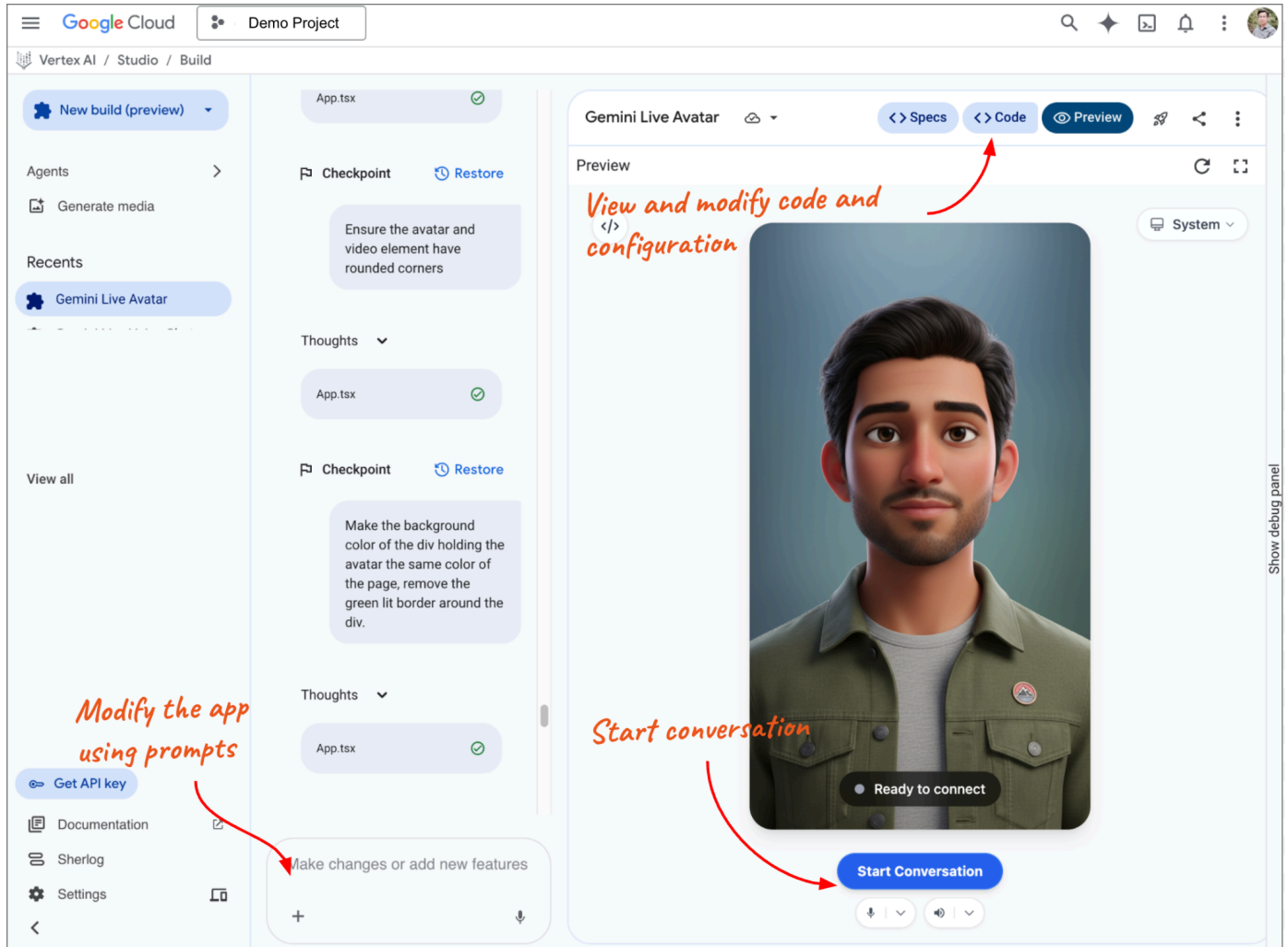
- **Access Vertex AI Studio:** Navigate to <https://console.cloud.google.com/vertex-ai>.
- **Create a Live API session:** Select the **Live API** from the **New** session dropdown.



- **Model Selection:** Select a live avatar-supporting model **gemini-3.1-flash-live-preview-04-2026**.
- **Action Configuration:** Select the Live Avatar action card; the avatar selection panel will render on the right prompt configuration panel.



- **Avatar and Persona Setup:** Pick a built-in avatar and optionally define other parameters like system instructions or the voice name.
- **Deployment:** Select 'Start Session' to navigate to the Vertex App Builder with a prebuilt application.
- **Interaction:** Play around and start a conversation with the avatar to test the conversational experience.



- **[Optional] Vibe Code Customization:** In the App Builder, you can
 - View or modify the API configuration in `resources/live_service_configuration.ts` in the Code.
 - Input prompts in the chat to modify application features. Example Prompts:
 - "Change the primary theme color of the app to a deep forest green and update all button borders to be more rounded."
 - "Add a toggle switch in the settings panel that allows users to enable or disable the display of the live transcript."
 - "Modify the avatar display area to be circular instead of rectangular and add a subtle glowing border effect when the avatar is speaking."

Note: The pre-built app is portable. It can be saved, downloaded, and deployed to Cloud Run.

API Configuration & Implementation

Enabling Live Avatar

Currently, Live Avatar is supported by model `gemini-3.1-flash-live-preview-04-2026`.

To instruct the Live API to generate an avatar video, you must configure the session during the initial connection setup:

1. Set `response_modalities` (or `responseModalities`) in your generation configuration to `["VIDEO"]`.
2. Provide an `avatar_config` (or `avatarConfig`) to specify the visual appearance.
3. Configure the voice using `speech_config.voice_config` (or `speechConfig.voiceConfig`).

Sample code (Websocket example):

```
Python
GENERATION_CONFIG = {
    "responseModalities": ["VIDEO"],
    "speechConfig": {"voiceConfig": {"prebuiltVoiceConfig": {"voiceName": "Puck"}}}
}

AVATAR_CONFIG = {"avatar_name": "Ben"}
...
async with websockets.connect(URI, additional_headers=headers) as ws:
    setup_msg = {
        "setup": {
            "model": MODEL,
            "generationConfig": GENERATION_CONFIG,
            "avatarConfig": AVATAR_CONFIG,
        }
    }
    await ws.send(json.dumps(setup_msg))
    await ws.recv()
...
```

Configuring the Avatar

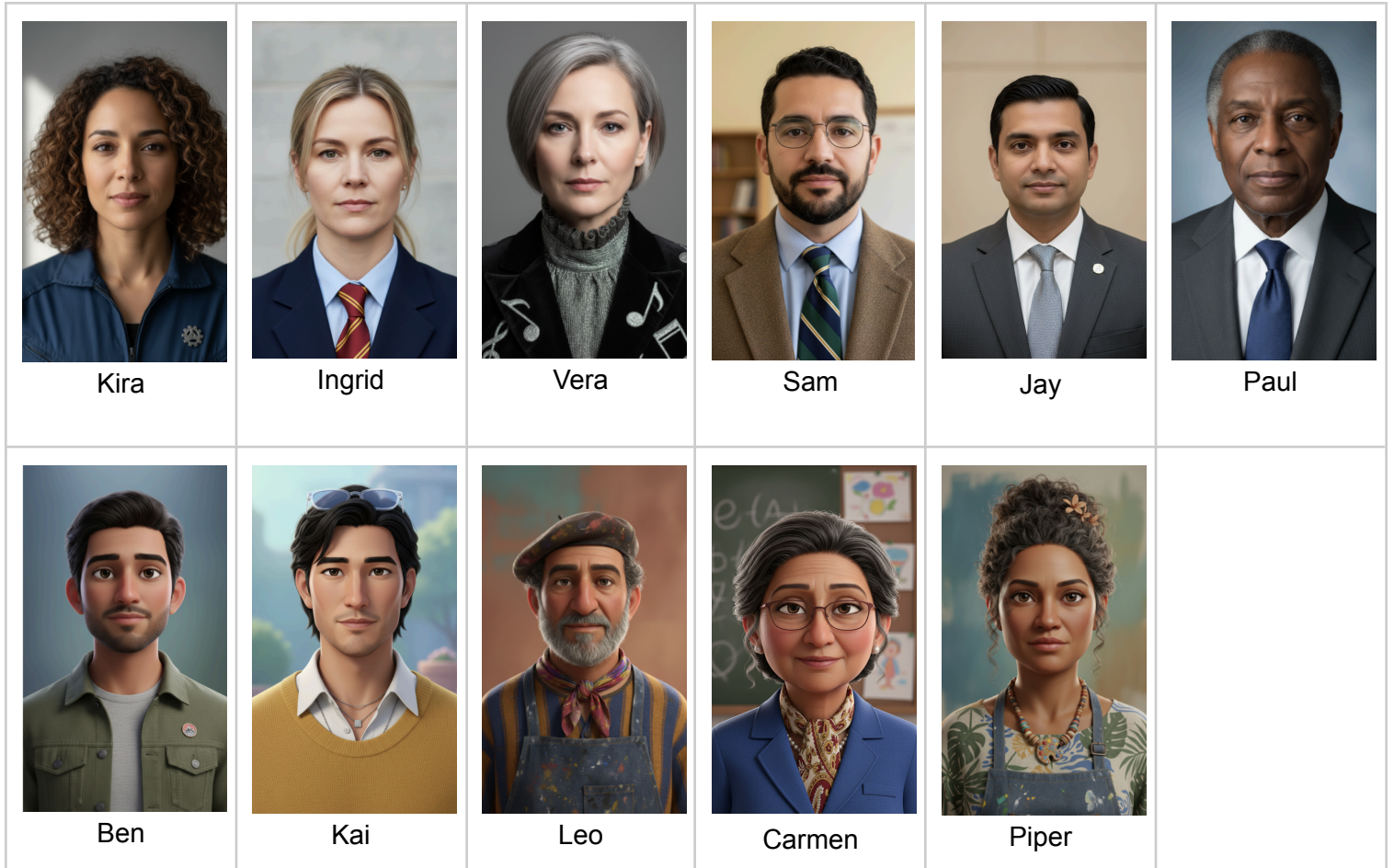
Using Built-in Avatars

To use a pre-existing avatar, simply specify its name in the `avatar_config`.

```
Python
AVATAR_CONFIG = {
    "avatar_name": "Ben"
}
```

Available Built-in Avatars

- **Professional/Realistic Styles:** Kira, Ingrid, Vera, Sam, Jay, Paul
- **Stylized/Character Styles:** Ben, Kai, Carmen, Leo, Piper



Configuring the Voice

Using Built-in Voices

You can pair your avatar with one of Gemini Live API's default voices.

```
Python
CONFIG = {
    "response_modalities": ["VIDEO"],
    "speech_config": {
        "voice_config": {
            "prebuilt_voice_config": {"voice_name": "Puck"}
        }
    }
}
```

Gemini Live API supports the following 30 voice options:

Zephyr -- <i>Bright</i>	Puck -- <i>Upbeat</i>	Charon -- <i>Informative</i>
Kore -- <i>Firm</i>	Fenrir -- <i>Excitable</i>	Leda -- <i>Youthful</i>
Orus -- <i>Firm</i>	Aoede -- <i>Breezy</i>	Callirrhoe -- <i>Easy-going</i>
Autonoe -- <i>Bright</i>	Enceladus -- <i>Breathy</i>	Iapetus -- <i>Clear</i>
Umbriel -- <i>Easy-going</i>	Algieba -- <i>Smooth</i>	Despina -- <i>Smooth</i>
Erinome -- <i>Clear</i>	Algenib -- <i>Gravelly</i>	Rasalgethi -- <i>Informative</i>
Laomedeia -- <i>Upbeat</i>	Achernar -- <i>Soft</i>	Alnilam -- <i>Firm</i>
Schedar -- <i>Even</i>	Gacrux -- <i>Mature</i>	Pulcherrima -- <i>Forward</i>
Achird -- <i>Friendly</i>	Zubenelgenubi -- <i>Casual</i>	Vindemiatrix -- <i>Gentle</i>
Sadachbia -- <i>Lively</i>	Sadaltager -- <i>Knowledgeable</i>	Sulafat -- <i>Warm</i>

Stream Data Formats

	Input	Output
Audio	Raw 16-bit PCM audio at 16kHz, little-endian	Raw 16-bit PCM audio at 24kHz, little-endian
Video	Frame Format: JPEG encoded images (bytes). Frame Rate: 1 frame per second (1 FPS). Encoding: Base64 encoded JPEG data. Optimal Resolution: Native \768 \times 768\ pixels. Mime Type: image/jpeg	Mime Type: video/mp4

Implementation Examples

Using WebSocket

This example demonstrates how to establish a session, send the setup configuration, and process the incoming video frames using the standard WebSockets approach.

```

Python
import asyncio
import base64
import json
import websockets

# Configuration
PROJECT_ID = "your-project-id"
LOCATION = "us-central1"
MODEL_ID = "gemini-3.1-flash-live-preview-04-2026"
MODEL = f"projects/{PROJECT_ID}/locations/{LOCATION}/publishers/google/models/{MODEL_ID}"

```

```

HOST = f"{LOCATION}-autopush-aiplatform.sandbox.googleapis.com"
URI = f"wss://{HOST}/ws/google.cloud.aiplatform.v1.LlmBidiService/BidiGenerateContent"

headers = {
    "Authorization": f"Bearer YOUR_ACCESS_TOKEN",
    "Content-Type": "application/json"
}

CONFIG = {
    "responseModalities": ["VIDEO"],
    "speechConfig": {
        "voiceConfig": {
            "prebuiltVoiceConfig": {"voiceName": "Puck"}
        }
    }
}

AVATAR_CONFIG = {
    "avatar_name": "Ben"
}

async def main():
    async with websockets.connect(URI, additional_headers=headers) as ws:

        # 1. Send the Session Setup message
        setup_msg = {
            "setup": {
                "systemInstruction": {"parts": [{"text": "You are a helpful AI assistant."}]},
                "model": MODEL,
                "generationConfig": CONFIG,
                "avatarConfig": AVATAR_CONFIG,
            }
        }
        await ws.send(json.dumps(setup_msg))
        await ws.recv() # Wait for setup acknowledgment

        # 2. Send the User's input
        user_msg = {
            "realtime_input": {
                "text": "Hi, can you introduce yourself?"
            }
        }
        await ws.send(json.dumps(user_msg))

        video_data = []

```

```

# 3. Process the server's streaming response
async for message in ws:
    response = json.loads(message)
    parts = response.get("serverContent", {}).get("modelTurn", {}).get("parts", [])

    # Extract inline video frame data
    for part in parts:
        if "inlineData" in part:
            # Decode base64 video frames and append to the buffer
            video_data.append(base64.b64decode(part["inlineData"]["data"]))

    # Check if the turn is complete
    server_content = response.get("serverContent")
    is_turn_complete = server_content is not None and
server_content.get("turnComplete")

    if is_turn_complete:
        if video_data:
            output_file = "output_video.mp4"
            with open(output_file, "wb") as f:
                f.write(b"".join(video_data))
            print("Video saved successfully.")
        break

asyncio.run(main())

```

Using Gen AI SDK

This example demonstrates how to establish a session, send the setup configuration, and process the incoming video frames using the Gen AI SDK for Python.

Install the Google Gen AI SDK

Live Avatar requires Gen AI SDK for Python version **1.72.0** or later.

Shell

```
pip install --upgrade google-genai
```

Example

Replace the **PROJECT_ID** value with your Google Cloud Project ID.

Python

```

# Initialize the GenAI Client
client = genai.Client(vertexai=True, project=PROJECT_ID, location="us-central1")

```

```

# Configure the LiveConnect Session for VIDEO modality, avatar and voice
config = types.LiveConnectConfig(
    response_modalities=["VIDEO"],
    system_instruction="You are a helpful AI assistant.",
    avatar_config=types.AvatarConfig(avatar_name="Kai"),
    speech_config=types.SpeechConfig(
        voice_config=types.VoiceConfig(
            prebuilt_voice_config=types.PrebuiltVoiceConfig(
                voice_name="Puck"
            )
        )
    )
)

async def main_sdk():
    print("Connecting to the Gemini Live API via the Google Gen AI SDK...")

    # Establish a live connect session
    async with client.aio.live.connect(model=MODEL_ID, config=config) as session:
        print("Session established. Sending text message...")

        await session.send_realtime_input(text="Hi, are you there?")

        video_data = []

        async for message in session.receive():
            if message.server_content:
                if message.server_content.interrupted:
                    continue
                # Accumulate the video parts from the model's turn
                model_turn = message.server_content.model_turn
                if model_turn and model_turn.parts:
                    for part in model_turn.parts:
                        if part.inline_data:
                            # The SDK already decodes the binary base64 data for us
                            video_data.append(part.inline_data.data)

                # Check if this turn has completed
                if message.server_content.turn_complete:
                    print(f"Turn complete. Total video frames received: {len(video_data)}")
                    if video_data:
                        output_file = "output_video_sdk.mp4"
                        with open(output_file, "wb") as f:
                            f.write(b"".join(video_data))
                        display(Video(output_file, embed=True, width=600, height=400,
html_attributes="controls autoplay"))
                    break

```

```
await main_sdk()
```

Important API Changes

When integrating the Avatar feature with the latest Gemini Live API, be aware of the following general API functionality changes:

- Media Chunks Deprecation: The `media_chunks` field in `BidiGenerateContentRealtimeInput` is being deprecated. Clients should now use the specific audio, video, and text fields within `BidiGenerateContentRealtimeInput` to send corresponding modality inputs.
- Client Content Repurposing: The `BidiGenerateContentClientContent` message is repurposed specifically to seed initial context history.
- For built-in tool use, only Google Search is supported in the latest Gemini Live API. Code Execution and Vertex AI RAG engine are not yet supported
- Only synchronous function calling is supported in the latest Gemini Live API.

Additional Resources

- [Intro to Live Avatar Notebook](#)